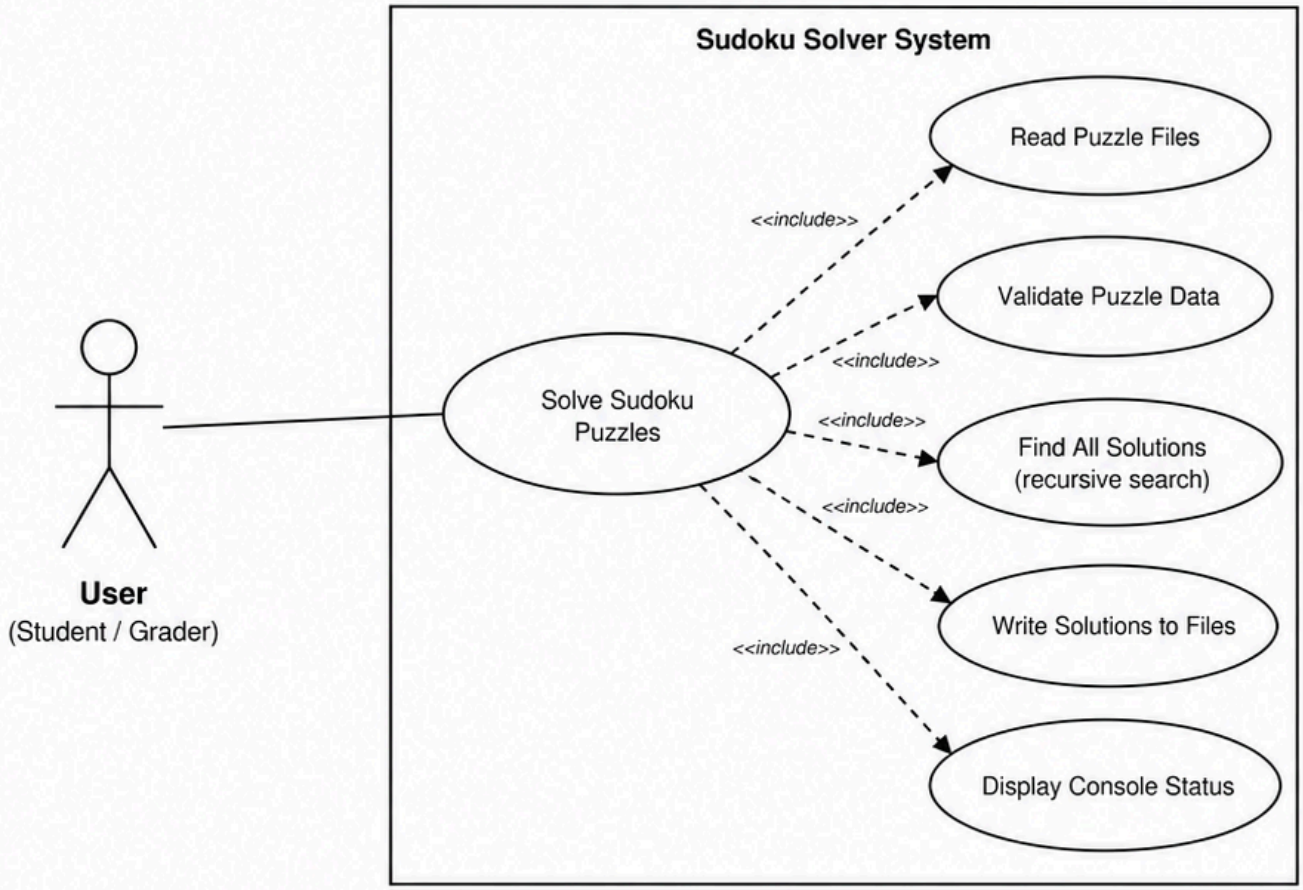
EECS 348 Assignment 5 - UML Use Case Diagram

Name: Charvi Reddy Konudula **Revised by:** Rithvik Kondapalli **Date:** October 29, 2026

1. Identify the use case

Use case: Solve Sudoku Puzzles. The actor is the **User** (a student running the program or a grader). The system is the revised object-oriented Sudoku Solver program (Assignment5.cpp).



2. Brief description

The user runs the program. For each of the five puzzle files (puzzle1.txt through puzzle5.txt) the system reads the puzzle, checks that it is valid, uses recursive backtracking to find every solution, writes the results to a matching solution file, and prints a short status message to the console.

3. Step-by-step flow of events (outline)

1. The user runs Assignment5.exe.
2. The system starts with puzzle1.txt and prepares solution1.txt.
3. The system reads the 81 cells from the file (digits 1-9 and _ for blanks).
4. The system validates the data (file opens, 81 cells, only _ or 1-9, and no conflicting clues).
5. The system opens the corresponding solution file for output.

EECS 348 Assignment 5 - UML Use Case Diagram

Original Author: Charvi Reddy Konudula Revised by: Rithvik Kondapalli Oct 29, 2026

3. Step-by-step flow of events (outline) (continued)

6. The system searches recursively for valid digits and backtracks when needed.
7. When a complete grid is found, the system writes that solution to the solution file and continues searching for more solutions.
8. If no complete solution is found, the system writes "No solution found." to the solution file.
9. After finishing a puzzle, the system writes "Solutions found: n" when solutions exist and prints a status message to the console.
10. The system repeats steps 2-9 for puzzle2.txt through puzzle5.txt, then ends.

4. Detailed description

Use case name	Solve Sudoku Puzzles
Actor	User (student or grader)
Goal	Find every valid solution for each of the five Sudoku puzzle files and write the results to matching output files.
Trigger	The user runs Assignment5.exe.
Preconditions	Assignment5.exe and puzzle1.txt through puzzle5.txt are in the same folder. Each puzzle file holds 9 rows of 9 space-separated symbols, using 1-9 for clues and _ for blanks.
Postconditions (success)	For every puzzle, the corresponding solution file contains either all solutions followed by "Solutions found: n" or the single line "No solution found." The console reports the result for each puzzle file.
Main flow	Steps 1-10 in section 3.
Alternative flow A: multiple solutions	If a puzzle has more than one solution (for example puzzle5.txt), the system writes each solution in turn, followed by the total number of solutions found.
Alternative flow B: no solution	If every candidate digit fails (for example puzzle4.txt), the search ends with no complete grid and the system writes "No solution found." to the output file.
Exception: file cannot be opened	The system prints an error naming the file and moves on to the next puzzle.
Exception: invalid data	If a file has fewer than 81 cells, more than 81 cells, an invalid symbol, or conflicting clues, the system prints an error describing the problem and moves on to the next puzzle.
Included use cases	Read Puzzle Files; Validate Puzzle Data; Find All Solutions (recursive search with backtracking); Write Solutions to Files; Display Console Status.
Rules and constraints	Each digit 1-9 appears exactly once in every row, every column, and every 3x3 block. The search must be recursive and object-oriented (a SudokuSolver class).
Frequency of use	Once per run of the program; the use case repeats for each of the five puzzle files.